

Pairwise global alignment & Needleman-Wunsch algorithm

Pairwise sequence alignment

Goal: Find the best alignment of two sequences.

Given two strings:

- What is their overall level of similarity?
- Which parts of them are similar?

```
X = A C C C G A T A  
Y = T A C C C G A C C T
```

Reminder: common types of genetic variation


Mismatch
...GCGTGGGTACGCTTA**T**TCGTCAAGCTTTAGCGT...
...GCGTGGGTACGCTTA**A**TCGTCAAGCTTTAGCGT...

Single nucleotide variant (SNV) /
single nucleotide polymorphism (SNP)


Gap
...GGTCAAGC--TAGCGT...
...GGTCAAGCTTTAGCGT...

Insertion or deletion
(indel)

Global pairwise alignment

Alignment of the full lengths of two strings X , Y .

Formal definition:

Pairwise global alignment = A modification $X \rightarrow X'$, $Y \rightarrow Y'$ done by insertions of gap symbols (-) into X and Y , where every character in X' is aligned to a character in Y' .

$X = A C C C G A T A$
 $Y = T A C C C G A C C T$

$X' = - - A C C C G A T A$
 $Y' = T A C C C G A C C T$

$X' = - A C C C G A - - T A$
 $Y' = T A C C C G A C C T -$

Global pairwise alignment

Details of $X \rightarrow X'$, $Y \rightarrow Y'$ modification:

- The original order of characters in X and Y is preserved.
- The length of X' is the same as the length of Y' .
- A gap character in X' cannot be aligned to a gap character in Y' .

$X = A C C C G A T A$
 $Y = T A C C C G A C C T$

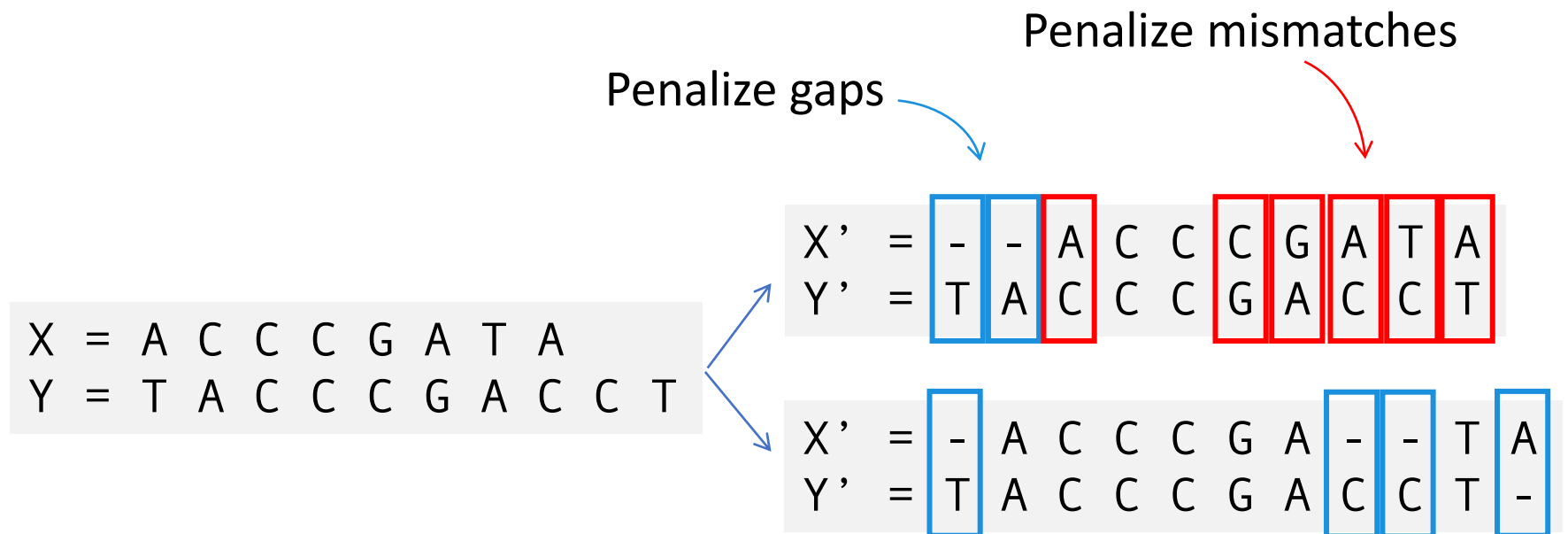
$X' = - - A C C C G A T A$
 $Y' = T A C C C G A C C T$

$X' = - A C C C G A - - T A$
 $Y' = T A C C C G A C C T -$

How to compare two alignments?

Assume positions are independent.

Total score = SUM(score of each indel) + SUM(score of each pair of aligned characters)



How to compare two alignments?

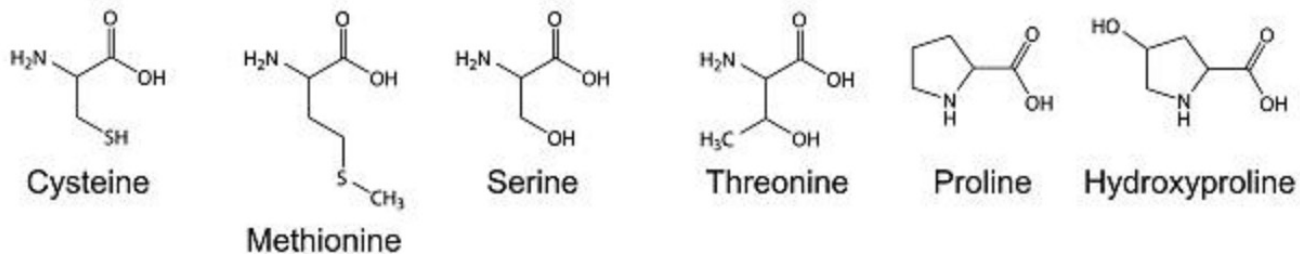
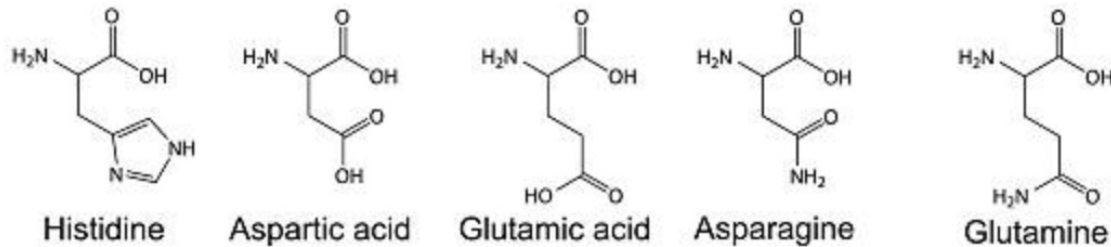
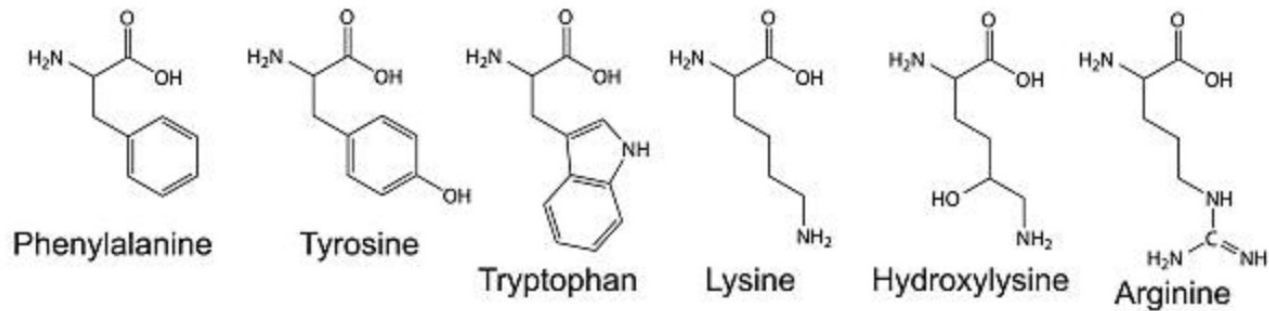
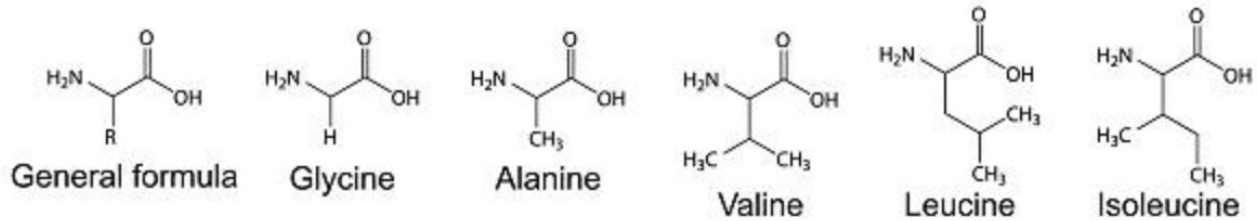
Scoring aligned characters:

- For example: +2 for a match, -1 for a mismatch

σ	A	C	T	G
A	2	-1	-1	-1
C		2	-1	-1
T			2	-1
G				2

- Should we penalize some mismatches more than others?

Reminder: Amino acid structures



How to compare two alignments?

Scoring aligned characters:

- Should we penalize some mismatches more than others?

Look at sequences of the same protein from many species:



Human	W	Q	S	T	A	R	W	L	R	R	L	V	F	Q	H	S	R	A	W	P	L	L	Q	T	F	A	F	S	A	W	W	H	G
Pig	W	H	S	T	A	Q	W	L	R	R	L	V	F	Q	Q	G	R	T	W	P	L	L	Q	T	F	V	F	S	A	W	W	H	G
Cow	W	Q	S	T	A	R	W	L	R	R	L	V	F	Q	Q	R	R	T	W	P	L	L	Q	T	F	L	F	S	A	W	W	H	G
Dog	W	Q	S	T	A	R	W	L	R	R	L	V	F	Q	Q	R	R	T	W	P	L	L	Q	T	F	L	F	S	A	W	W	H	G
Rat	W	R	S	T	A	Q	W	L	K	R	L	V	F	Q	R	S	R	R	W	P	V	L	Q	T	F	A	F	S	A	W	W	H	G
Mouse	W	R	S	T	A	L	W	L	R	R	L	V	F	R	K	S	R	R	W	P	L	L	Q	T	F	A	F	S	A	W	W	H	G
Chicken	W	R	S	T	S	L	W	L	R	R	L	V	F	Q	R	C	P	V	Q	P	L	L	A	T	F	A	F	S	A	W	W	H	G
Zebrafish	W	Q	T	T	V	D	W	L	R	K	I	V	F	N	R	T	S	R	S	P	L	F	M	T	F	G	F	S	A	L	W	H	G

How to compare two alignments?

Scoring aligned characters:

- Should we penalize some mismatches more than others?

Some amino acid substitutions are seen frequently in evolution, because the amino acids are biologically similar.

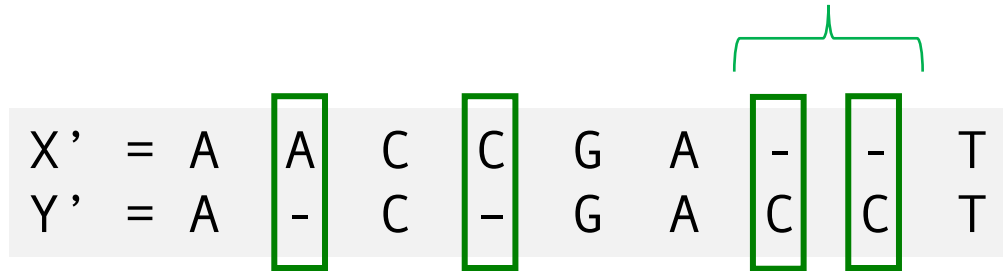
$$\sum_{\substack{c_1, c_2 \\ \text{aligned}}} \sigma(c_1, c_2)$$

← Each score depends on the two aligned characters c_1, c_2

How to compare two alignments?

Scoring gaps:

- How should we score consecutive gaps?



X'	=	A	A	C	C	G	A	-	-	T
Y'	=	A	-	C	-	G	A	C	C	T

- Two independent events?
- One event of length two?

How to compare two alignments?

Scoring gaps:

1) Linear penalty = (number of indels * penalty-factor)

Linear Gap Penalty = $4W_s$

		W_s		W_s			W_s	W_s		
X'	=	A	A	C	C	G	A	-	-	T
Y'	=	A	-	C	-	G	A	C	C	T

2) Affine penalty = Gap-open-penalty +
(number of indels * penalty-factor)

Affine Gap Penalty = $3W_g + 4W_s$

		$W_g + W_s$		$W_g + W_s$			$W_g + 2W_s$			
X'	=	A	A	C	C	G	A	-	-	T
Y'	=	A	-	C	-	G	A	C	C	T

How to compare two alignments?

Scoring scheme with linear gap penalty (W_s, σ):

$$\text{Score of alignment} = \sum_{\substack{c_1, c_2 \\ \text{aligned}}} \sigma(c_1, c_2) + n_{\text{gaps}} * W_s \quad \sigma: \Sigma \times \Sigma \rightarrow \mathbb{R}$$

$$\text{Score of alignment} = \sum_{\substack{c_1, c_2 \\ \text{aligned}}} \sigma(c_1, c_2) \quad \sigma: \{\Sigma, "-"\} \times \{\Sigma, "-"\} \rightarrow \mathbb{R}$$

How to compare two alignments?

$$\text{Score of alignment} = \sum_{\substack{c_1, c_2 \\ \text{aligned}}} \sigma(c_1, c_2)$$

$$\sigma: \{\Sigma, "-"\} \times \{\Sigma, "-"\} \rightarrow \mathbb{R}$$

σ	A	C	T	G	-
A	2	-1	-1	-1	-1
C		2	-1	-1	-1
T			2	-1	-1
G				2	-1
-					

$X' = - \quad - \quad A \quad C$
 $Y' = A \quad A \quad C \quad C$

Score = -1

$X' = - \quad A \quad - \quad C$
 $Y' = A \quad A \quad C \quad C$

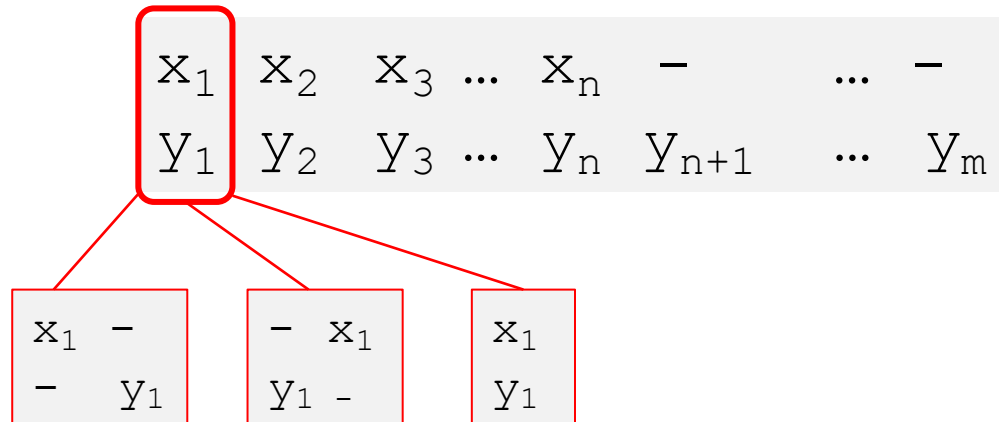
Score = +2

Global pairwise alignment

Goal: Given two strings X and Y , find their highest scoring global alignment.

Number of possible alignments is exponential:

$$X = x_1 \dots x_n, \quad Y = y_1, \dots, y_m$$



Global pairwise alignment

Number of alignments is exponential in $\min\{n, m\}$.

But can we find the best alignment in polynomial time?

Strategy: Make use of **optimal sub-structure**.

(i.e. the solution can be derived from optimal solutions to sub-problems)

Optimal sub-structure

Input strings: $(x_1 \dots x_n)$, $(y_1 \dots y_m)$

Let V_{ij} = value of the optimal alignment of the sub-strings
 $(x_1 \dots x_i)$, $(y_1 \dots y_j)$

Claim (optimal sub-structure):

V_{ij} can be derived immediately from
 $\{V_{i-1,j} , V_{i,j-1} , V_{i-1,j-1} \}$

Approach: Use **dynamic programming**.

Optimal sub-structure

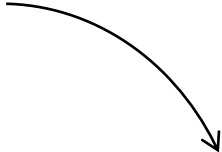
Dynamic programming approach:

- Consider the V matrix.
- Induction:
 - Fill in the base case (with no sub-problems).
 - Gradually fill the rest of the matrix based on the solutions for the preceding sub-problems.
- Value of the optimal alignment of the full strings X and Y is given by V_{nm} .

Needleman-Wunsch algorithm

X = - A C T
Y = A A G T

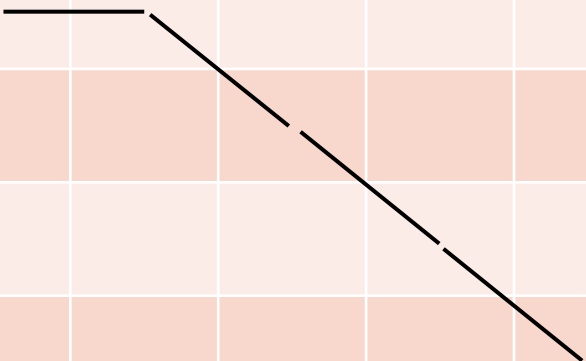
How is this alignment represented?



Y = A A G T

X = A C T

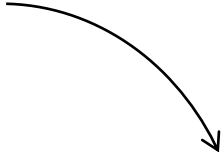
			A	A	G	T
	(i, j)	0	1	2	3	4
	0					
A	1					
C	2					
T	3					



Needleman-Wunsch algorithm

X = A C - T
Y = A A G T

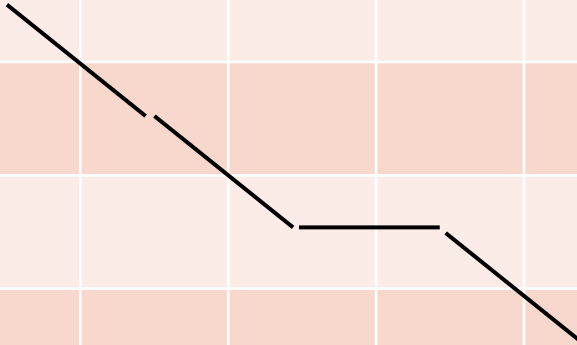
How is this alignment represented?



Y = A A G T

X = A C T

			A	A	G	T
	(i, j)	0	1	2	3	4
	0					
A	1					
C	2					
T	3					



Needleman-Wunsch algorithm

Base case: $V(0, 0) = 0$

X = -

Y = -

			A	A	G	T
	(i, j)	0	1	2	3	4
	0	0				
A	1					
C	2					
T	3					

Needleman-Wunsch algorithm

Base case: $V(0, 1) = W_s$

X = -
Y = A

			A	A	G	T
	(i, j)	0	1	2	3	4
	0	0	W_s			
A	1					
C	2					
T	3					

Needleman-Wunsch algorithm

Base case: $V(0, 2) = 2W_s$

X = - -
Y = A A

			A	A	G	T
	(i, j)	0	1	2	3	4
	0	0	W_s	$2W_s$		
A	1					
C	2					
T	3					

Needleman-Wunsch algorithm

Base case: $0 < j \leq m$:

$$V(0, j) = \sum_{k=1 \dots j} \sigma(-, Y_k) = W_s * j$$

X = - - -
Y = A A G

X = - - - -
Y = A A G T

			A	A	G	T
	(i, j)	0	1	2	3	4
	0	0	W_s	$2W_s$	$3W_s$	$4W_s$
A	1					
C	2					
T	3					

Needleman-Wunsch algorithm

Base case: $0 < i \leq n$:

$$V(i, 0) = \sum_{k=1 \dots i} \sigma(X_k, -) = W_s * k$$

X = A

Y = -

X = A C

Y = - -

X = A C T

Y = - - -

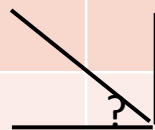
			A	A	G	T
	(i, j)	0	1	2	3	4
	0	0	W_s	$2W_s$	$3W_s$	$4W_s$
A	1	W_s				
C	2	$2W_s$				
T	3	$3W_s$				

Needleman-Wunsch algorithm

Recursion: $V(i, j) = ?$

Three possibilities for what $V(i, j)$ represents.

			A	A	G	T
	(i, j)	0	1	2	3	4
	0	0	W_S	$2W_S$	$3W_S$	$4W_S$
A	1	W_S				
C	2	$2W_S$				
T	3	$3W_S$				



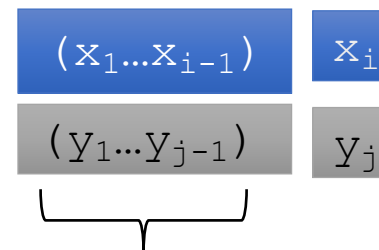
Needleman-Wunsch algorithm

Recursion: $V(i, j) = ?$

Remember, $V(i, j)$ = value of the optimal alignment of the sub-strings $(x_1 \dots x_i)$, $(y_1 \dots y_j)$.

Three options at location $V(i, j)$:

1. Alignment ends with x_i matched with y_j



Alignment of $(x_1 \dots x_{i-1})$, $(y_1 \dots y_{j-1})$

$$V(i-1, j-1) + \sigma(x_i, y_j)$$

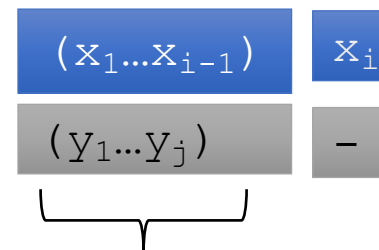
Needleman-Wunsch algorithm

Recursion: $V(i, j) = ?$

Remember, $V(i, j)$ = value of the optimal alignment of the sub-strings $(x_1 \dots x_i)$, $(y_1 \dots y_j)$.

Three options at location $V(i, j)$:

2. Alignment ends with x_i matched with gap



Alignment of $(x_1 \dots x_{i-1})$, $(y_1 \dots y_j)$

$$V(i-1, j) + \sigma(x_i, -)$$

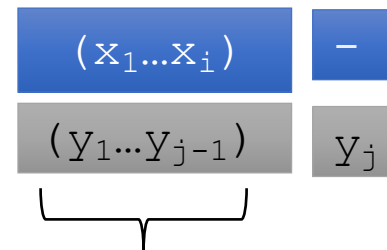
Needleman-Wunsch algorithm

Recursion: $V(i, j) = ?$

Remember, $V(i, j)$ = value of the optimal alignment of the sub-strings $(x_1 \dots x_i)$, $(y_1 \dots y_j)$.

Three options at location $V(i, j)$:

3. Alignment ends with y_j matched with gap



Alignment of $(x_1 \dots x_i)$, $(y_1 \dots y_{j-1})$

$$V(i, j-1) + \sigma(-, y_j)$$

Needleman-Wunsch algorithm

$$V(i, j) = \text{Max} \left\{ \begin{array}{l} V(i-1, j-1) + \sigma(x_i, y_j) \\ V(i-1, j) + \sigma(x_i, -) \\ V(i, j-1) + \sigma(-, y_j) \end{array} \right.$$

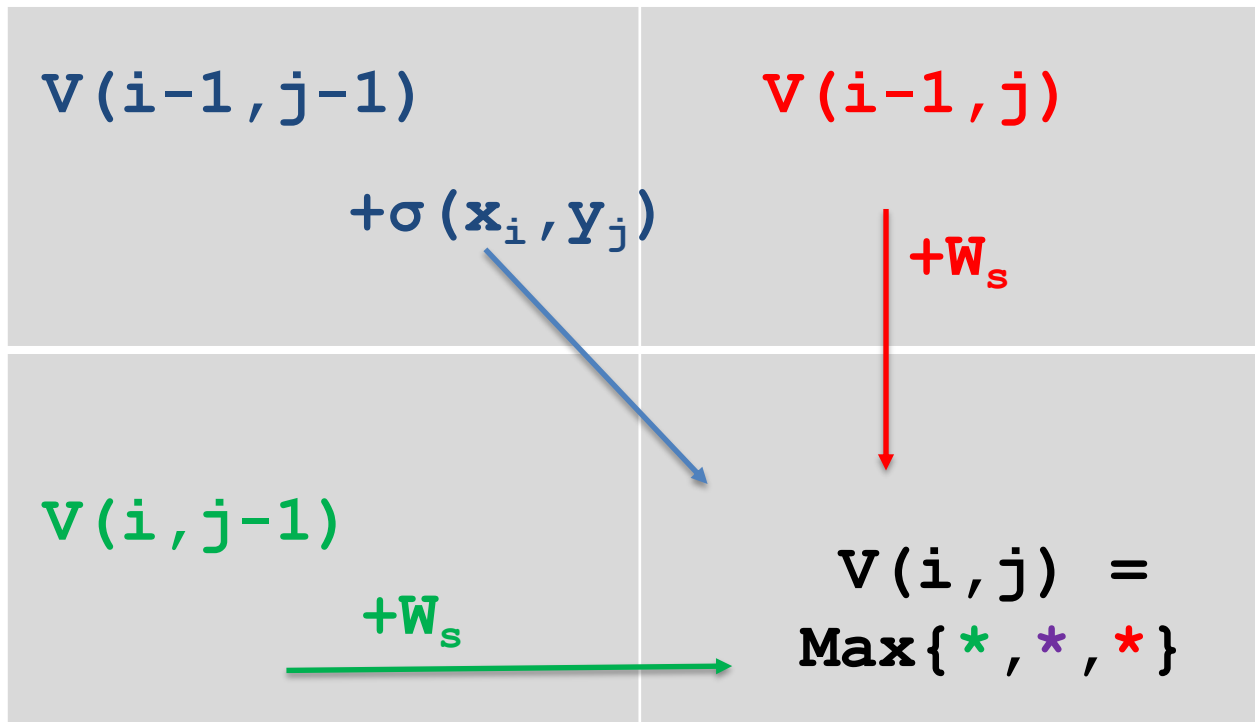
Optimal Alignment of $(x_1 \dots x_{i-1}), (y_1 \dots y_{j-1})$

Optimal Alignment of $(x_1 \dots x_{i-1}), (y_1 \dots y_j)$

Optimal Alignment of $(x_1 \dots x_i), (y_1 \dots y_{j-1})$

Needleman-Wunsch algorithm

Recursion: $V(i, j) = ?$



Needleman-Wunsch algorithm

Input: Strings $(x_1 \dots x_n)$, $(y_1 \dots y_m)$
Scoring function $\sigma: \{\Sigma, "-"\} \times \{\Sigma, "-"\} \rightarrow \mathbb{R}$

V_{ij} = value of optimal alignment for the strings $(x_1 \dots x_i)$, $(y_1 \dots y_j)$
Desired output: V_{nm}

Initialization:

$$V(0, 0) = 0$$
$$0 < i \leq n: V(i, 0) = \sum_{k=1 \dots i} \sigma(x_k, -)$$
$$0 < j \leq m: V(0, j) = \sum_{k=1 \dots j} \sigma(-, y_k)$$

Recursion:

$$V(i, j) = \text{Max} \begin{cases} V(i-1, j-1) + \sigma(x_i, y_j) \\ V(i-1, j) + \sigma(x_i, -) \\ V(i, j-1) + \sigma(-, y_j) \end{cases}$$

Initialization:

$$V(0, 0) = 0$$

$$0 < i \leq n: \quad V(i, 0) = \sum_{k=1 \dots i} \sigma(X_k, -)$$

$$0 < j \leq m: \quad V(0, j) = \sum_{k=1 \dots j} \sigma(-, Y_k)$$

$X = A \quad C \quad T$
 $Y = A \quad A \quad G \quad T$

σ	A	C	T	G	-
A	2	-1	-1	-1	-1
C		2	-1	-1	-1
T			2	-1	-1
G				2	-1
-					

Recursion:

$$V(i, j) = \text{Max} \begin{cases} V(i-1, j-1) + \sigma(x_i, y_j) \\ V(i-1, j) + \sigma(x_i, -) \\ V(i, j-1) + \sigma(-, y_j) \end{cases}$$

			A	A	G	T
	(i, j)	0	1	2	3	4
	0					
A	1					
C	2					
T	3					

Initialization:

$$V(0, 0) = 0$$

$$0 < i \leq n: \quad V(i, 0) = \sum_{k=1 \dots i} \sigma(X_k, -)$$

$$0 < j \leq m: \quad V(0, j) = \sum_{k=1 \dots j} \sigma(-, Y_k)$$

X = A C T
Y = A A G T

σ	A	C	T	G	-
A	2	-1	-1	-1	-1
C		2	-1	-1	-1
T			2	-1	-1
G				2	-1
-					

Recursion:

$$V(i, j) = \text{Max} \begin{cases} V(i-1, j-1) + \sigma(x_i, y_j) \\ V(i-1, j) + \sigma(x_i, -) \\ V(i, j-1) + \sigma(-, y_j) \end{cases}$$

			A	A	G	T
	(i, j)	0	1	2	3	4
	0	0	-1	-2	-3	-4
A	1	-1				
C	2	-2				
T	3	-3				

$X = A$ C T
 $Y = A$ A G T

σ	A	C	T	G	-
A	2	-1	-1	-1	-1
C		2	-1	-1	-1
T			2	-1	-1
G				2	-1
-					

$V(1,1) = \text{Max}$

$$V(0,0) + \sigma(A,A)$$

$$= 0 + 2 = 2$$

$$V(0,1) + \sigma(A,-)$$

$$= -1 - 1 = -2$$

$$V(1,0) + \sigma(-,A)$$

$$= 1 - 1 = -2$$

empty

A

empty

A

-

A

A

-

A

-

-

A

			A	A	G	T
	(i,j)	0	1	2	3	4
	0	0	-1	-2	-3	-4
A	1	-1	2			
C	2	-2				
T	3	-3				

$$V(1,2) = \text{Max}$$

$$V(0,1) + \sigma(A,A)$$

$$= -1 + 2 = 1$$

$$V(0,2) + \sigma(A,-)$$

$$= -2 - 1 = -3$$

$$V(1,1) + \sigma(-,A)$$

$$= 2 - 1 = 1$$

-

A

A

A

--

A

A A

-

A

-

A

A

X = A C T
Y = A A G T

σ	A	C	T	G	-
A	2	-1	-1	-1	-1
C		2	-1	-1	-1
T			2	-1	-1
G				2	-1
-					

			A	A	G	T
	(i,j)	0	1	2	3	4
	0	0	-1	-2	-3	-4
A	1	-1	2	1		
C	2	-2				
T	3	-3				

Initialization:

$$V(0, 0) = 0$$

$$0 < i \leq n: \quad V(i, 0) = \sum_{k=1 \dots i} \sigma(X_k, -)$$

$$0 < j \leq m: \quad V(0, j) = \sum_{k=1 \dots j} \sigma(-, Y_k)$$

X = A C T
Y = A A G T

σ	A	C	T	G	-
A	2	-1	-1	-1	-1
C		2	-1	-1	-1
T			2	-1	-1
G				2	-1
-					

Recursion:

$$V(i, j) = \text{Max} \begin{cases} V(i-1, j-1) + \sigma(x_i, y_j) \\ V(i-1, j) + \sigma(x_i, -) \\ V(i, j-1) + \sigma(-, y_j) \end{cases}$$

			A	A	G	T
	(i, j)	0	1	2	3	4
	0	0	-1	-2	-3	-4
A	1	-1	2	1	0	-1
C	2	-2	1	1	0	-1
T	3	-3	0	0	0	2

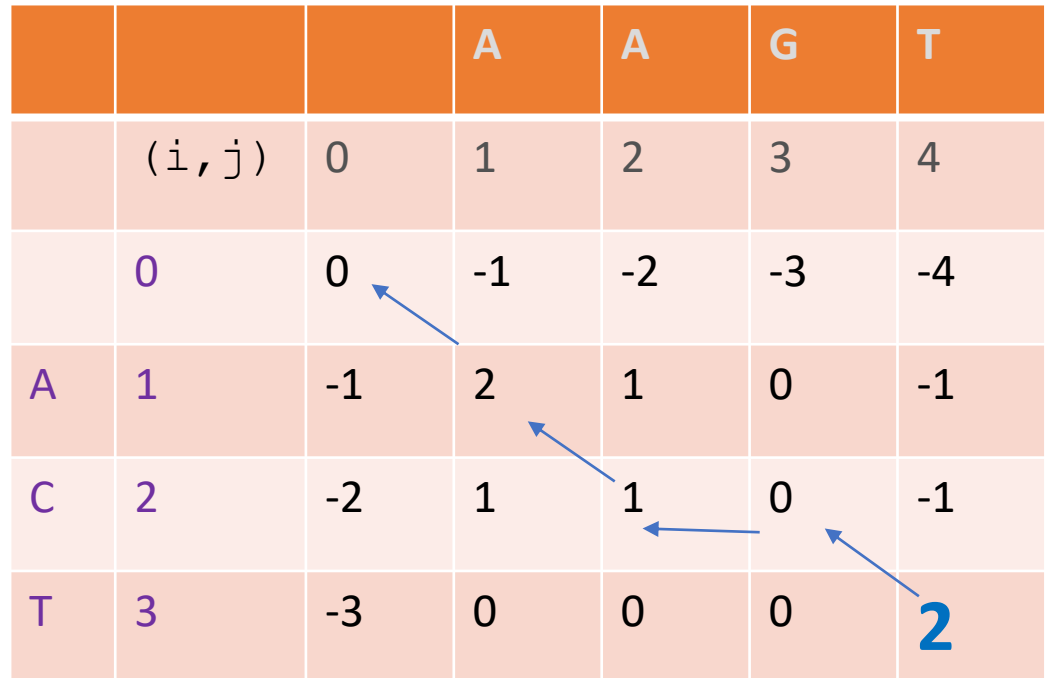
Traceback to find optimal alignment

Now we have the score of the optimal alignment, $V(n, m)$.

Q: How do we determine the optimal alignment itself?

A: Store pointers

			A	A	G	T
	(i, j)	0	1	2	3	4
	0	0	-1	-2	-3	-4
A	1	-1	2	1	0	-1
C	2	-2	1	1	0	-1
T	3	-3	0	0	0	2



Traceback to find optimal alignment

Trace the pointers back from the bottom right corner:

X = A C T
Y = A A G T

X' = T
Y' = T

σ	A	C	T	G	-
A	2	-1	-1	-1	-1
C		2	-1	-1	-1
T			2	-1	-1
G				2	-1
-					

			A	A	G	T
	(i, j)	0	1	2	3	4
	0	0	-1	-2	-3	-4
A	1	-1	2	1	0	-1
C	2	-2	1	1	0	-1
T	3	-3	0	0	0	2

Traceback to find optimal alignment

Trace the pointers back from the bottom right corner:

X = A C T
Y = A A **G** T

X' = - T
Y' = **G** T

σ	A	C	T	G	-
A	2	-1	-1	-1	-1
C		2	-1	-1	-1
T			2	-1	-1
G				2	-1
-					

			A	A	G	T
	(i, j)	0	1	2	3	4
	0	0	-1	-2	-3	-4
A	1	-1	2	1	0	-1
C	2	-2	1	1	0	-1
T	3	-3	0	0	0	2

Traceback to find optimal alignment

Trace the pointers back from the bottom right corner:

X = A C T
Y = A A G T

X' = C - T
Y' = A G T

σ	A	C	T	G	-
A	2	-1	-1	-1	-1
C		2	-1	-1	-1
T			2	-1	-1
G				2	-1
-					

			A	A	G	T
	(i, j)	0	1	2	3	4
	0	0	-1	-2	-3	-4
A	1	-1	2	1	0	-1
C	2	-2	1	1	0	-1
T	3	-3	0	0	0	2

Traceback to find optimal alignment

Trace the pointers back from the bottom right corner:

X = A C T
Y = A A G T

X' = A C - T
Y' = A A G T

σ	A	C	T	G	-
A	2	-1	-1	-1	-1
C		2	-1	-1	-1
T			2	-1	-1
G				2	-1
-					

			A	A	G	T
	(i, j)	0	1	2	3	4
	0	0	-1	-2	-3	-4
A	1	-1	2	1	0	-1
C	2	-2	1	1	0	-1
T	3	-3	0	0	0	2

Traceback to find optimal alignment

$$\text{Score} = 2 - 1 - 1 + 2 = 2$$

X = A C T
Y = A A G T

X' = A C - T
Y' = A A G T

σ	A	C	T	G	-
A	2	-1	-1	-1	-1
C		2	-1	-1	-1
T			2	-1	-1
G				2	-1
-					

			A	A	G	T
	(i, j)	0	1	2	3	4
	0	0	-1	-2	-3	-4
A	1	-1	2	1	0	-1
C	2	-2	1	1	0	-1
T	3	-3	0	0	0	2

Traceback to find optimal alignment

$X = A \quad C \quad T$
 $Y = A \quad A \quad G \quad T$

σ	A	C	T	G	-
A	2	-1	-1	-1	-1
C		2	-1	-1	-1
T			2	-1	-1
G				2	-1
-					

$X' = - \quad A \quad C \quad T$
 $Y' = A \quad A \quad G \quad T$

$X' = A \quad - \quad C \quad T$
 $Y' = A \quad A \quad G \quad T$

$X' = A \quad C \quad - \quad T$
 $Y' = A \quad A \quad G \quad T$

Equal scores

			A	A	G	T
	(i, j)	0	1	2	3	4
0	0	0	-1	-2	-3	-4
A	1	-1	2	1	0	-1
C	2	-2	1	1	0	-1
T	3	-3	0	0	0	2

Summary of Needleman-Wunsch

Dynamic programming algorithm (uses optimal sub-structure).

Assumes linear gap penalty.

Run time? $O(nm)$ or $O(n^2)$ assuming $n=O(m)$

Space requirements?

Also $O(n^2)$ to store pointers for traceback